This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

SpikeTrack: A Spike-driven Framework for Efficient Visual Tracking

Qiuyang Zhang[1], Jiujun Cheng[1,†], Qichao Mao[1,†], Cong Liu[2], Yu Fang[1], Yuhong Li[3], Mengying Ge[4], Shangce Gao[5]

[1]Tongji University [2]Nova University of Lisbon [3]Stockholm University
[4]Shanghai University [5]University of Toyama

Abstract

Spiking Neural Networks (SNNs) promise energy-efficient vision, but applying them to RGB visual tracking remains difficult: Existing SNN tracking frameworks either do not fully align with spike-driven computation or do not fully leverage neurons' spatiotemporal dynamics, leading to a trade-off between efficiency and accuracy. To address this, we introduce SpikeTrack, a spike-driven framework for energy-efficient RGB object tracking. SpikeTrack employs a novel asymmetric design that uses asymmetric timestep expansion and unidirectional information flow, harnessing spatiotemporal dynamics while cutting computation. To ensure effective unidirectional information transfer between branches, we design a memory-retrieval module inspired by neural inference mechanisms. This module recurrently queries a compact memory initialized by the template to retrieve target cues and sharpen target perception over time. Extensive experiments demonstrate that SpikeTrack achieves the state-of-the-art among SNN-based trackers and remains competitive with advanced ANN trackers. Notably, it surpasses TransT on LaSOT dataset while consuming only 1/26 of its energy. To our knowledge, SpikeTrack is the first spike-driven framework to make RGB tracking both accurate and energy efficient. The code and models are available at this URL.

1. Introduction

Spiking neural networks (SNNs) are a promising energy-efficient computing paradigm that simulates the spatiotemporal dynamics and spiking mechanisms of biological neurons [20]. Their spike-driven computation has two advantages: (i) computation is triggered only when driven by events [21], and (ii) matrix multiplications between spike tensors and weights can be converted into sparse additions [7]. This gives SNNs a significant power-saving advantage over ANNs on neuromorphic chips [24, 25]. SNNs have shown strong results on multiple vision tasks [13, 17, 41], and their spatiotemporal dynamics make them natural candidates for tracking continuously moving objects.

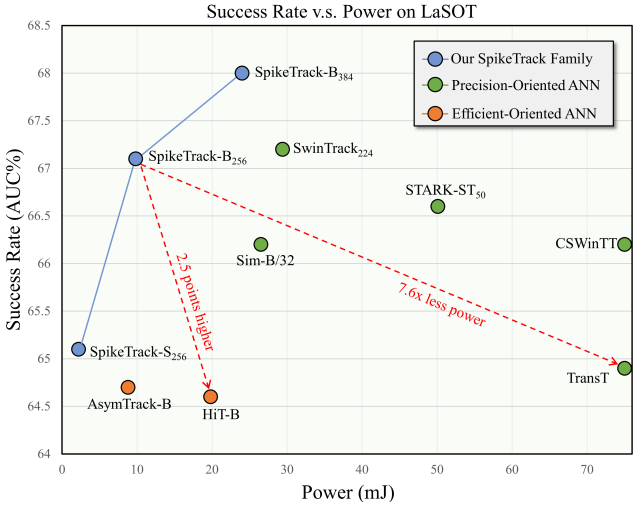


Figure 1. Energy–accuracy trade-off on LaSOT[5]. SpikeTrack achieves lower energy consumption than efficient ANN trackers while matching the accuracy of precision-oriented methods.

Current SNN tracking work falls into RGB-based and event-based methods. Within the RGB-based line, SiamSNN [18] and Spike-SiamFC++ [32], adopt the Siamese architecture, achieve tracking via network conversion and end-to-end training, respectively. Although these methods use spiking neurons in form, they decode spike signals into continuous values for computation, preventing fully spike-driven processing and reducing energy efficiency. Event-based methods [26, 34] adapt the dense interaction framework [2, 37] from ANN, also known as the one-stream architecture, as shown in Fig. 2. This approach concatenates the search region and multiple templates along the token length dimension within a single timestep, feeding them into the backbone for joint modeling via spike self-attention. However, this direct imitation

† Corresponding authors: Jiujun Cheng (chengjj@tongji.edu.cn), Qichao Mao (mao_qichao@tongji.edu.cn).

underuses the spatiotemporal associative dynamic of SNNs, and dense, bidirectional interactions greatly increase computational overhead. This raises a research question: Can we design an SNN that adheres to the spike-driven paradigm while fully leveraging spatiotemporal modeling capabilities for efficient RGB tracking?

To address this problem, we propose SpikeTrack, a spike-driven SNN for energy-efficient RGB tracking. SpikeTrack adopts an asymmetric Siamese architecture, with asymmetric timestep inputs and unidirectional information transfer, as shown in Fig. 2. Specifically, the template branch expands across multiple timesteps, assigning a template to each step and jointly modeling template representations through neuron's spatiotemporal dynamics, while the search branch performs efficient single-timestep inference. Information flows only from the template branch to the search branch, allowing the computation-heavy template branch to run only during initialization or template updates, thereby cutting computation. Additionally, to ensure effective unidirectional information transfer between branches, we design a memory-retrieval module (MRM) inspired by neural inference mechanism [27]. This module recurrently queries a compact memory initialized from the template features to retrieve target cues and sharpen target perception over time.

Extensive experiments demonstrate that SpikeTrack achieves strong energy efficiency and accuracy with a simple framework, outperforming prior SNN-based trackers. For instance, SpikeTrack-S_{256} outperforms SpikeSiamFC++ by 8.5% on the UAV123 dataset. Moreover, as shown in Fig. 1, SpikeTrack-S_{256} surpasses the efficiency-oriented AsymTrack [39] with 2.5× better energy efficiency, while SpikeTrack-B_{256} outperforms the precision-oriented TransT [3] with 7.6× energy savings and 2.2% higher accuracy.

Our main contributions are summarized as follows:

- We design an asymmetric SNN that fully utilizes the spatiotemporal dynamics of neuron while significantly reducing computational cost.
- We propose a brain-inspired memory retrieval module that enables effective unidirectional information transfer.
- Building on the above designs, we propose SpikeTrack, a spike-driven framework for efficient RGB-based tracking, with a family of model variants. Experiments across multiple benchmarks demonstrate its effectiveness.

2. Related Work

SNNs in Vision Tasks. Recently, SNN-based approaches have achieved performance comparable to ANNs across various vision tasks, including image classification [35, 36], object detection [17], semantic segmentation [13], and video classification [41], as well as higher-level applications such as autonomous driving perception [40] and embodied intelligence [10]. By modeling neuronal membrane potential dynamics, SNNs possess powerful spatiotemporal encoding capabilities, making them particularly promising for tracking tasks that require perceiving continuously moving objects.

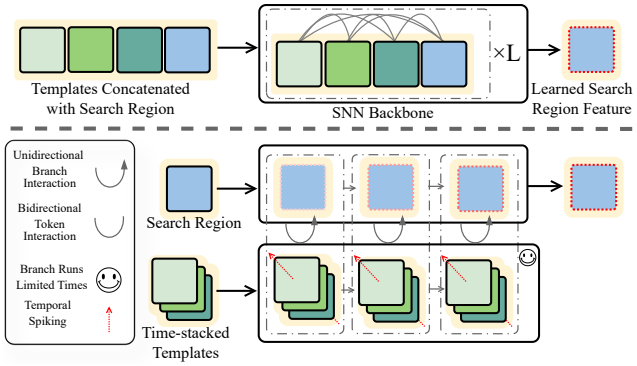


Figure 2. Structure comparison between one-stream tracking SNN (top) and our asymmetric tracking SNN (bottom). *L* represents the number of blocks in the backbone.

Visual Tracking Architecture. Visual tracking aims to predict a target's position and scale across video frames given its initial template. ANN-based trackers follow either two-stream (Siamese) or one-stream designs. Two-stream methods extract template and search features separately, then model their relation via cross-correlation or Transformer interaction. OSTrack [37] adopts a one-stream design, concatenating template and search patches in a Vision Transformer to jointly extract and relate features with strong results. However, AsymTrack [39] shows that such bidirectional interactions are costly on edge devices, and proposes an asymmetric Siamese network with unidirectional template modulation for competitive lightweight tracking. Inspired by this, we design an asymmetric architecture for RGB-based SNN tracking, using asymmetric timestep inputs and memory-retrieval-based unidirectional transfer to achieve efficient tracking with minimal overhead.

SNN-based Visual Tracking. Current SNN-based tracking research primarily targets event-camera inputs, where sparse event data and one-stream architectures yield strong results [26, 34], but the reliance on dedicated hardware limits its practical adoption. RGB-based tracking offers a more deployable alternative; however, existing efforts such as SiameseSNN [18] and SpikeSiameseFC++ [32] are tied to specific ANN frameworks, suffering from poor scalability, limited performance, and lacking comprehensive evaluation or energy analysis. To address these issues, we propose SpikeTrack, a concise and efficient RGB tracking baseline with extensive benchmark evaluation and detailed theoretical energy analysis.

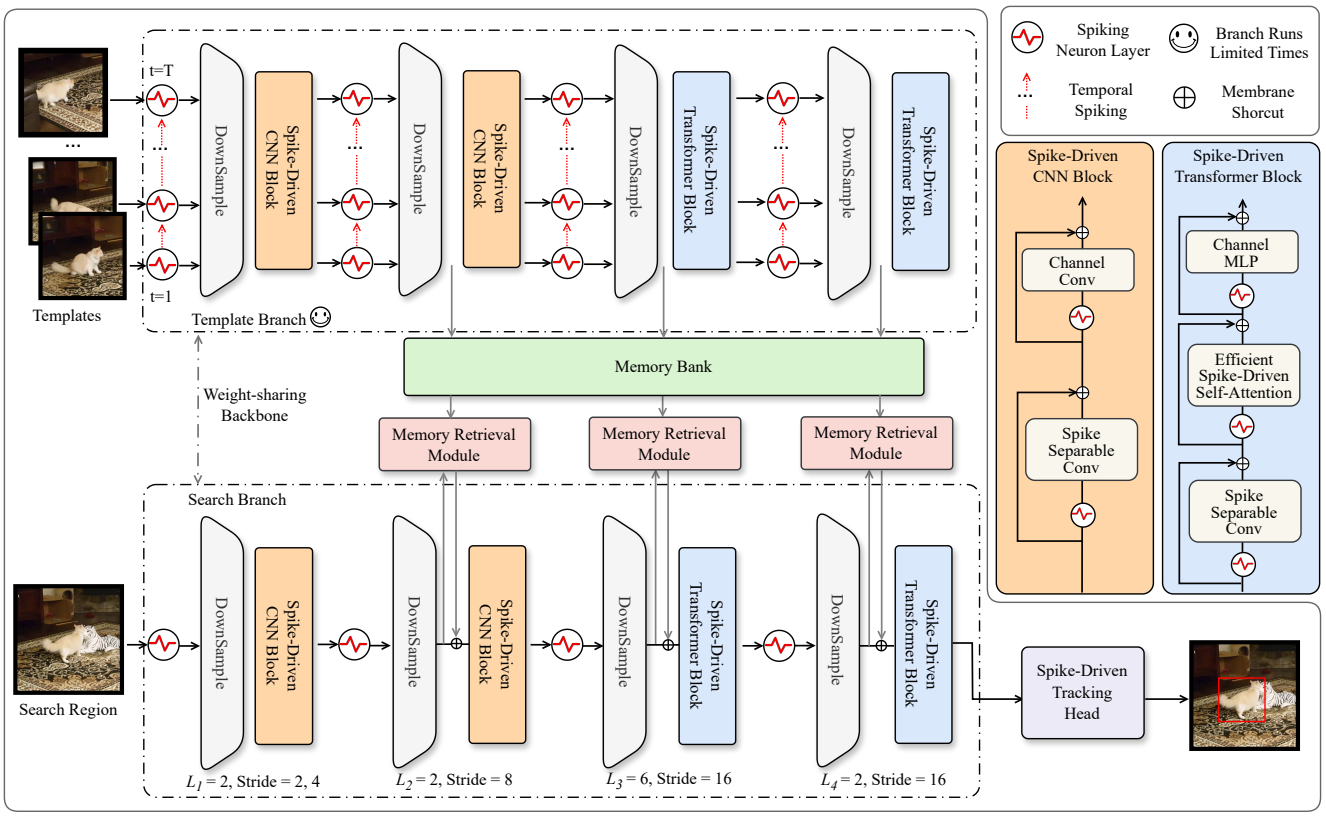


Figure 3. Overview of SpikeTrack. The network consists of three components: a weight-sharing siamese backbone, a memory retrieval module for information transfer, and a prediction head. We use asymmetric timestep inputs and unidirectional information flow. During inference, template branch features are converted and cached as memory. The search branch queries this memory to extract target cues. The template branch runs once, per initialization or update.

3. SpikeTrack-based Visual Tracking

In this section, we present the proposed SpiketTrack in detail. We begin with Sec. 3.1, briefly describing the overall network architecture, followed by Sec. 3.2, which introduces the basic spiking neuron model used. Subsequently, Sec. 3.3 describe the network components in detail. Finally, Sec. 3.4 introduces the training and inference pipeline.

3.1. Overview

As shown in Fig. 3, SpikeTrack comprises three components: a shared-weight spiking backbone, a memory retrieval module (MRM) for unidirectional branch interaction, and a prediction head. During inference, the template branch performs inference once after template initialization or template update, caching features from different intermediate layers in the memory bank as memory. The search branch then uses MRMs to retrieve the targets cues from memory and progressively refine target perception. Finally, the prediction head consumes the region features to produce the tracking results.

3.2. Spiking Neuron Model

We adopt the Normalized Integer Leaky Integrate-and-Fire (NI-LIF) neuron [13]. It trains with normalized integer activations based on the classical LIF neuron [19], and converts integer activations into equivalent spikes during inference to preserve spike-driven characteristics. In this work, we design the leaky factor as a trainable variable to allow the network to adaptively model the correlation between timesteps. The neural dynamics equation for NI-LIF is:

$$U[t] = \beta_t H[t-1] + Y[t] \tag{1}$$

$$S[t] = \text{Clip}\big(\text{round}\big(U[t]\big), 0, D\big)/D \tag{2}$$

$$H[t] = U[t] - S[t] \times D \tag{3}$$

$$\beta_t = \sigma(\theta_t) \tag{4}$$

where t is the timestep, $U[t]$ represents the membrane potential after charging but before firing. Spatial input $Y[t]$ is extracted from the original spike input through a Conv or MLP operation, temporal input $\beta H[t-1]$ is derived from the decay of the membrane potential at the previous timestep, β_t is the leaky factor, $\sigma(\cdot)$ is the sigmoid function, θ_t is a learnable variable, $S[t]$ is the output spike, $H[t]$ is

the membrane potential after firing, $round(\cdot)$ is a round operation, $Clip(x, min, max)$ implies clipping the input x to $[min, max]$, and D is a hyper-parameter to emit the maximum integer value.

3.3. SpikeTrack Architecture

Asymmetric Siamese Backbone. We adopt Spike-Driven Transformer v3 [36] as the backbone. It is a meta-Transformer style SNN [38] composed of convolutional and Transformer-based SNN blocks. Specifically, the input of the backbone are template image $Z \in \mathbb{R}^{T\times 3\times H\times W}$, where T depends on the number of templates, and the search image $X \in \mathbb{R}^{3\times H\times W}$. The input images first pass through a 7×7 convolutional layer for $2\times$ downsampling, followed by a 3×3 convolutional layer after each stage for $2\times$ downsampling. Each stage contains L_i blocks, where the first two stages utilize CNN-based SNN blocks, and the latter two stages incorporate Transformer-based SNN blocks. In the template branch, from the second downsampling layer onward, features from each downsampling stage, the intermediate layer of stage3 ($l_3/2$) , and the final layer of stage 4 are initialized in the memory bank. When the search branch reaches layers aligned with the cached memory, the MRM retrieves target cues from the corresponding memory to enhance target perception. After the final stage, the enriched search feature are fed into the tracking head for prediction.

Next, we introduce the two basic components that make up the backbone:

CNN Block. Each CNN block consists of a spike separable convolution followed by a channel-wise convolution, as detailed below:

$$U' = U + \text{SSConv}(U), \tag{5}$$

$$U'' = U' + \text{ChannelConv}(U'), \tag{6}$$

$$\text{SSConv}(U) = \text{Conv}_{\text{pw}}(\mathcal{SN}(\text{Conv}_{\text{dw}}(\mathcal{SN}(\text{Conv}_{\text{pw}}(\mathcal{SN}(U)))))), \tag{7}$$

$$\text{ChannelConv}(U') = \text{Conv}(\mathcal{SN}(\text{Conv}(\mathcal{SN}(U')))), \tag{8}$$

where $\mathcal{SN}(\cdot)$ is the spike neuron layer, $\text{SSConv}(\cdot)$ is the spike separable convolution, $\text{Conv}(\cdot)$ is the vanilla convolution, $\text{Conv}_{\text{pw}}(\cdot)$ and $\text{Conv}_{\text{dw}}(\cdot)$ are the point-wise and depth-wise convolutions, respectively. The BN layers are omitted for brevity.

Transformer Block. Each Transformer block contains a separable convolution, an efficient spike-driven self-attention module (E-SDSA) and a channel MLP , as detailed below:

$$U' = U + \text{SSConv}(U), \tag{9}$$

$$U'' = U' + \text{E-SDSA}(U'), \tag{10}$$

$$U''' = U'' + \text{ChannelMLP}(U''), \tag{11}$$

$$\text{ChannelMLP}(U'') = \text{Linear}(\mathcal{SN}(\text{Linear}(\mathcal{SN}(U'')))), \tag{12}$$

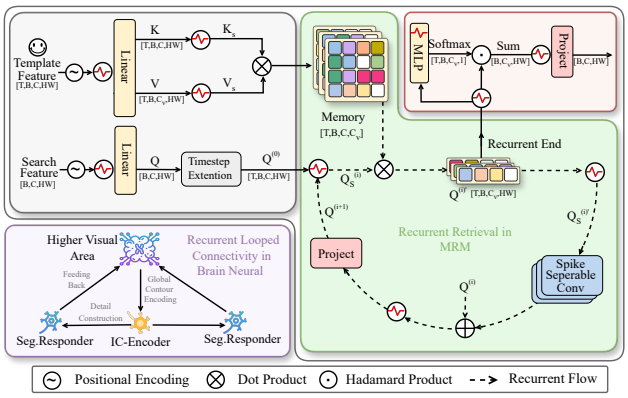


Figure 4. Implementation details of the Memory Retrieval Module. The purple legend (bottom left) illustrates the recurrent, looped connectivity structure in the brain. For simplicity of illustration, the temporal spiking across timesteps are omitted.

where E-SDSA$(\cdot)$ employs binary spiking tensors K_S, Q_S, $V_S \in \{0, 1\}^{N\times D}$ as the Query, Key and Value, respectively, where N is the token length and D is the channel size. The subscript S represents that the tensor is in spike form. By omitting the softmax function, E-SDSA enables computational order rearrangement to achieve linear complexity with respect to token length. The process is described as follows:

$$Q_S = \mathcal{SN}(\text{Linear}(U)), \tag{13}$$

$$K_S = \mathcal{SN}(\text{Linear}(U)), \tag{14}$$

$$V_S = \mathcal{SN}(\text{Linear}_\gamma(U)), \tag{15}$$

$$\begin{aligned} U' &= \text{Linear}_{\frac{1}{\gamma}}\mathcal{SN}(\underbrace{(Q_S K_S^T)V_S}_{\mathcal{O}(N^2D)} * scale) \\ &= \text{Linear}_{\frac{1}{\gamma}}\mathcal{SN}(\underbrace{Q_S(K_S^T V_S)}_{\mathcal{O}(ND^2)} * scale) \end{aligned} \tag{16}$$

where γ is the expansion factor of V_S, used to enhance the representation of E-SDSA, and the constant $scale$ factor is used for gradient stabilization.

Memory Retrieval Module (MRM). As illustrated in Fig. 4, the MRM enables effective unidirectional information transfer from the template to the search branch. Its design draws on neuroscientific findings [27] regarding visual perception under occlusion, where recurrent connectivity in the brain's V1 L2/3 area achieves complete perceptual inference through iterative refinement based on prior expectations—a mechanism naturally aligned with template-based tracking.

For efficiency, features entering the MRM are downsampled via average pooling to the resolution of the final back-

bone stage and upsampled back at the output. The template feature F_Z is projected to produce Key K_S and Value V_S, while the search feature F_X is temporally expanded to produce Query $Q_S^{(0)}$. Leveraging the linear complexity of spike-based attention (Eq. (16)), a memory matrix $M = K_S^T V_S$ is pre-computed once during template initialization and reused across frames. From this point, the recurrent loop begins.

The recurrent processing comprises three stages. First, *global contour encoding*: $Q_S^{(i)}$ retrieves from M via a scaled dot-product followed by a spiking neuron:

$$Q_S^{(i)\prime} = \mathcal{SN}(Q_S^{(i)} M \cdot scale). \tag{17}$$

Second, *detail construction*: $Q_S^{(i)\prime}$ is processed by T dedicated SSConvs along the temporal dimension, with each timestep assigned its own operator to improve sensitivity to temporal variations:

$$Q^{(i)\prime\prime} = \text{SSConv}_t(Q_S^{(i)\prime}[t]), \quad t \in \{1, 2, \ldots, T\}. \tag{18}$$

Third, *feedback refinement*: a residual connection with projection simulates feedback to higher-level visual areas:

$$Q_S^{(i+1)} = \text{Project}(\mathcal{SN}(Q^{(i)} + Q^{(i)\prime\prime})). \tag{19}$$

This process repeats for N iterations (N=1 for all variants). Finally, a spike-driven two-layer MLP generates channel-wise weights w for temporal fusion, and the result is projected back to the original channel dimensions:

$$F_{out} = \text{Project}\left(\sum_{t=1}^{T} w_t \odot Q_S^{(N)}[t]\right). \tag{20}$$

Prediction Head. We employ a center head to predict the object bounding box, following the design of OSTrack[37] while adopting a spike-driven mechanism. The feature of the search branch are passed through three parallel branches, each composed of several Conv-BN-NILIF layers. The last layer does not contain BN and NI-LIF. These branches predict (1) the target's center localization (classification), (2) the local offset induced by resolution reduction, and (3) the normalized bounding-box width and height.

3.4. Training objective and Inference

Trainning. We comine weighted focal loss [12], ℓ_1 loss and generalized IoU loss [23] as the training objective. The loss function can be formulated as:

$$\mathcal{L} = \mathcal{L}_{class} + \lambda_G \mathcal{L}_{IoU} + \lambda_{L_1} \mathcal{L}_1, \tag{21}$$

where $\mathcal{L}_{class}$ denotes the weighted focal loss used for classification, $\mathcal{L}_{IoU}$ represents the generalized IoU loss, ℓ_1 is the $\mathcal{L}_1$ regression loss, and λ_G = 2 and λ_{L_1} = 5 are the regularization parameters.

Inference. During the inference process, the template set is regarded as a queue and is updated in a first-in-first-out order while keeping the first initial template fixed. The update strategy follows standard practices [33], using two hyperparameters: a update interval and an update score threshold. The update operation is performed when the update interval arrives and the predicted quality score is higher than the threshold. All models use the same set of hyperparameters.

To reduce training burden and keep the network simple, SpikeTrack omits a separate quality scoring module and instead uses the localization branch score in the prediction head as the confidence score.

4. Experiments

4.1. Implementation Details

The SpikeTrack models are implemented using Python 3.12 with PyTorch 2.0.0 and trained on 8 NVIDIA 4090 GPUs.

Model. We develop six SpikeTrack model variants to balance power and accuracy, varying in backbone size (base/small), input resolution (256/384), and number of timesteps (1/3). We adopt Spike-Driven Transformer (SDT) V3-19M [36] as backbone for SpikeTrack-Base and SDTV3-5.1M for SpikeTrack-Small. The backbones are initialized with the Imagenet-1K [4] pre-trained parameters.

Training. We train on standard SOT datasets: COCO [15], LaSOT [5], TrackingNet [22] and GOT-10k [9] (excluding 1k sequences from the train split to align with the training data of other trackers). The total batch size is 128. Template and search images are generated by expanding target bounding boxes by a factor of 4. The AdamW [16] optimizer is used for training. All models use the same training strategy

For T=1 models, we train for 320 epochs using 60k image pairs per epoch. The learning rates are set to 4e-5 for the backbone and 4e-4 for the head and MRMs, with a weight decay of 1e-4. The learning rate is reduced by a factor of 10 after 240 epochs.

For T>1 models, the training data consist of image groups containing one search region and T templates. Starting from the pretrained T=1 SpikeTrack weights, we train for 60 epochs with learning rates of 4e-4 for the MRM and learnable decay factor, and 4e-5 for other modules. The learning rate is decreased by 10× after 30 epochs.

Inference. For simplicity, all models use the same set of hyperparameters. The online template update interval is set to 25, with an update confidence threshold of 0.7 by default. A Hanning window penalty is applied to incorporate positional prior information in tracking, following standard practices [3].

Energy evaluation. We compare SpikeTrack with SNN and ANN tracking methods, following the energy consumption evaluation criteria used in previous work [17, 35, 36].

Table 1. Comparison of performance and efficiency across tracking methods. *Para.* and *Pow.* denote parameters (M) and power (mJ), respectively; *T* and *D* represent the timestep and maximum integer value emitted during training. Results marked with * are trained solely on the GOT-10K set. All results are reported in percentage (%). The top two snn results are highlighted in **bold** and underlined, respectively.

	Method	Efficiency			TrackingNet			GOT-10k			LaSOT			LaSOT$_{ext}$		
		Para.	Pow.	T×D	AUC	P$_N$	P	AO	SR$_{50}$	SR$_{75}$	AUC	P$_N$	P	AUC	P$_N$	P
SNN	SpikeTrack-B$_{384}$	36.8	27.3	3×4	**82.0**	**87.6**	**80.7**	**73.1**	**84.0**	**69.9**	66.7	76.8	72.9	**47.6**	**58.5**	**54.5**
			20.0	1×4	81.8	87.1	80.4	70.7	81.0	67.0	**67.5**	**77.8**	**73.3**	47.3	57.8	54.1
	SpikeTrack-B$_{256}$	36.8	9.8	3×4	81.1	86.9	79.1	72.2	83.9	67.7	67.1	77.7	72.5	46.7	57.6	52.9
			8.1	1×4	80.1	85.4	77.6	69.6	80.6	66.2	66.6	77.2	71.6	46.0	56.7	52.0
	SpikeTrack-S$_{256}$	11.2	3.7	3×4	78.7	84.6	75.5	67.8	79.9	61.7	64.5	76.0	69.0	43.9	54.5	49.2
			2.8	1×4	77.9	83.6	74.8	67.2	78.8	60.9	65.1	76.4	69.6	43.3	53.4	48.2
	SiamSNN [18]	-	-	20	-	-	-	31.4	32.7	-	-	-	-	-	-	-
ANN	AsymTrack-B [39]	3.36	8.3	-	80.0	84.5	77.4	67.7	76.6	61.4	64.7	73.0	67.8	44.6	-	-
	HiT-B [11]	42.1	19.8	-	80.0	84.4	77.3	64.0	72.1	58.1	64.6	73.3	68.1	44.1	-	-
	CSWinTT [28]	25.1	75.4	-	81.9	86.7	79.5	69.4*	78.9*	65.4*	66.2	75.2	70.9	-	-	-
	OSTrack$_{256}$ [37]	-	98.9	-	83.1	87.8	82.0	71.0*	80.4*	68.2*	69.1	78.7	75.2	47.4	57.3	53.3
	SwinTrack$_{224}$ [14]	23	29.4	-	81.1	-	78.4	71.3*	81.9*	64.5*	67.2	-	70.8	47.6	-	53.9
	Sim-B/32 [2]	-	26.5	-	79.1	-	83.9	-	-	-	66.2	76.1	-	-	-	-
	STARK-ST$_{50}$ [33]	23.5	50.1	-	81.3	86.1	-	68.0*	77.7*	62.3*	66.6	-	-	-	-	-
	TransT [3]	17.9	75.2	-	81.4	86.7	80.3	72.3	82.4	68.2	64.9	73.8	69.0	-	-	-
	TrSiam [29]	-	-	-	78.1	82.9	72.7	67.3*	78.7*	58.6*	62.4	-	60.6	-	-	-

The ANN energy cost is calculated as:

$$E_{ANN} = \text{FLOPs} \times E_{MAC}, \tag{22}$$

while the SNN energy cost is defined as:

$$E_{SNN} = \text{FLOPs} \times E_{AC} \times SFR \times T \times D, \tag{23}$$

where SFR denotes the average spike firing rate, T is the number of timesteps, and D is the upper limit of integer activation during training. All values are based on 32-bit floating-point implementations in 45nm technology [8], where E_{MAC} = 4.6 pJ and E_{AC} = 0.9 pJ. More evaluation details are presented in the Supplementary Material.

For SpikeTrack energy analysis, we define the spike firing rate as the average spike rate measured on LaSOT and GOT-10K. The template-branch energy is estimated by dividing its total energy by the update interval.

4.2. Tracker Comparisons

We compare our SpikeTrack with SNN trackers and ANN trackers on seven widely used tracking benchmarks.

GOT-10K [9]. GOT-10k test set contains 180 videos covering a wide range of common challenges in tracking. As reported in Tab. 1, SpikeTrack-S$_{256}$-T3 achieves a comparable AO score to the state-of-the-art efficient ANN AsymTrack-B while consuming only half the energy. In addition, SpikeTrack-B$_{256}$-T1 delivers a 38.2% AO improvement over the existing SNN tracker SiamSNN.

LaSOT [5]. LaSOT is a large-scale long-term tracking benchmark. The test set contains 280 videos with an average length of 2448 frames. The results on LaSOT are presented in Tab. 1. SpikeTrack-B$_{256}$-T3 surpasses TransT by 2.2% in AUC, yet requires less than one-seventh of its energy consumption. When comparing different SpikeTrack variants, the performance of S256 and B384 models does not improve as T increases. We attribute this to the higher demand for template precision in long-term tracking that our simple scoring mechanism introduces certain low-quality templates during updates, which in turn undermines predictive accuracy.

LaSOT$_{ext}$ [6]. LaSOT$_{ext}$ is a recently released dataset with 150 video sequences and 15 object classes. Across this dataset, as shown in Tab. 1, SpikeTrack variants follow the expected pattern: both higher T values and increased input resolution lead to gradual performance gains. Notably, SpikeTrack-B$_{256}$-T1 achieves a 1.4% higher AUC than AsymTrack-B while consuming less energy.

TrackingNet [22]. TrackingNet is a large-scale dataset containing 511 videos, which covers diverse object categories and scenes. As reported in Tab. 1, when matched in AUC score with SwinTrack$_{224}$, SpikeTrack-B$_{256}$-T3 oper-

Table 2. Performance comparison of different tracking methods. All evaluation metrics are based on Success Rate (AUC) and reported in percentage (%). The top two snn results are highlighted in **bold** and underlined, respectively.

Method	T×D	Pow.	TNL2K	OTB	UAV
SpikeTrack-B_{384}	3×4	27.3	**54.8**	68.9	**68.3**
	1×4	20.0	54.2	68.3	67.2
SpikeTrack-B_{256}	3×4	9.8	54.0	**69.6**	67.8
	1×4	8.1	53.3	68.0	67.5
SpikeTrack-S_{256}	3×4	3.7	52.0	69.4	66.2
	1×4	2.8	51.2	67.1	66.3
SpikeSiamFC++ [32]	1	-	-	64.4	57.8
SiamSNN [18]	20	-	-	49.3	-
AsymTrack-B [39]	-	8.3	-	-	66.5
HiT-B [11]	-	19.8	-	-	65.6
$OSTrack_{256}$ [37]	-	98.9	54.3	-	68.3
$SwinTrack_{224}$ [14]	-	29.4	53.0	-	-
Sim-B/32 [2]	-	26.5	51.1	-	-
STARK-ST_{50} [33]	-	50.1	54.3	-	68.2
TransT [3]	-	75.2	50.7	69.4	68.1
TrSiam [29]	-	-	-	70.8	67.4

ates at only one-third of the energy cost. SpikeTrack-B_{384}-T3 reaches an equivalent AUC with just 35% of the energy consumption of CSWinTT.

TNL2K [30]. TNL2K is a recently released large-scale dataset with 700 challenging video sequences. As shown in Tab. 2, against the strong one-stream ANN baseline $OSTrack_{256}$, SpikeTrack-B_{384}-T3 delivers 0.5% higher AUC while using less than one-third of its energy. Similarly, SpikeTrack-S_{256}-T1 achieves 0.5% better AUC than TransT with a mere 3% of the energy requirement.

UAV123 [1] and OTB100 [31]. Both of these are small-scale benchmarks including 123 and 100 videos respectively. The results on these two datasets are presented in Tab. 2. On the OTB dataset, SpikeTrack-S_{256}-T3 outperforms existing SNN-based methods SpikeSiamFC++ and SiamSNN by 5% and 20.1% AUC, respectively. For UAV123, SpikeTrack-B_{256}-T3 delivers 10% higher AUC compared to the best previousSNN results.

4.3. Ablation and Analysis.

As shown in Tab. 3, we ablate training methods, architecture design, and hyperparameter settings. For a fair comparison with the one-stream architecture, the baseline (#1) is a SpikeTrack-B_{256}-T2 trained from scratch, rather than (#2), which fine-tunes SpikeTrack-B_{256}-T1. Fine-tuning (#2) requires fewer epochs and performs better than training from scratch (#1).

Table 3. Ablation Study on LaSOT and GOT-10k. △ denotes the performance change (averaged over benchmarks) compared with the baseline. * indicates the non-spike version.

#	Method	Pow.	GOT-10K	LaSOT	△
1	Baseline	8.7	71.3	66.8	-
2	Fine-tuning	8.7	71.6	66.9	+0.2
3	One-stream	22.8	70.8	65.4	-0.8
4a	Vanilla Cross-attn	7.6	70.9	65.0	-1.1
4b	Modulation	6.8	58.3	49.9	-14.9
4c	Modulation*	10.2	64.0	58.1	-8.0
5	Mean Fusion	8.5	71.0	66.2	-0.4
6	Fixed Decay	8.9	68.9	66.0	-1.9
7	Smaller Template	8.3	69.6	65.7	-1.4

Asymmetric $v.s.$ **One-stream.** Following the structure in [26, 34] and keeping the same training settings, we compare the one-stream architecture with our asymmetric architecture, as shown in Tab. 3 (#3). Our method achieves better results with lower energy consumption. This shows that modeling templates with spatiotemporal neuron dynamics, combined with a memory retrieval module, outperforms jointly modeling all templates and search regions using with a backbone .

Effectiveness of the MRM. We replace the MRM with vanilla spike cross-attention, enabling search region features to learn from concatenated template features. As shown in Tab. 3 (#4a), this modification eliminates spatiotemporal processing and loop operations, reducing energy consumption but causing a noticeable drop in accuracy compared to the baseline. Furthermore, we replace the MRM with AsymTrack[39]'s template modulation module, implementing both spike and non-spike versions. (#4b) notes that the method is very lightweight after spiking conversion, but suffers severe performance degradation. The hybrid structure of (#4c) improves performance but remains suboptimal. This indicates that using templates as convolutional kernels for signal modulation is unsuitable for the coarse-grained representation of spiking networks.

Effectiveness of the Fusion Module. Table 3 (#5) compares the commonly used time-step averaging fusion method in SNN structures [17, 35, 36] with our proposed channel-wise weighted fusion method. The latter performs better.

Learnable Decay $v.s.$ **Fixed Decay.** As shown in Tab. 3 (#6), we compare the fixed membrane potential leaky factor used in previous SNN works [13, 17, 35, 36] with our learnable one. The learnable factor enables more flexible and controllable interactions across timesteps.

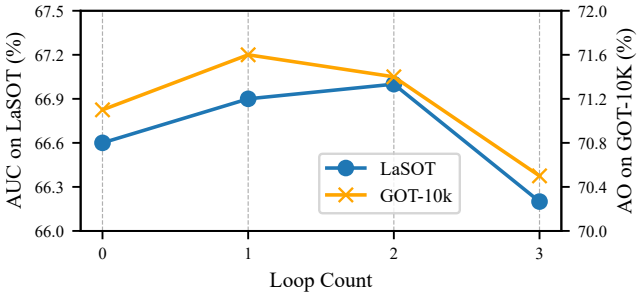


Figure 5. Influence of the number of retrieval loop in MRM.

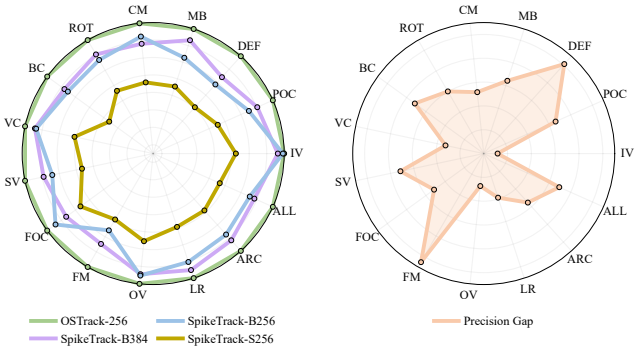


Figure 6. Gap analysis between SpikeTrack and a precision-oriented ANN across LaSOT attributes.

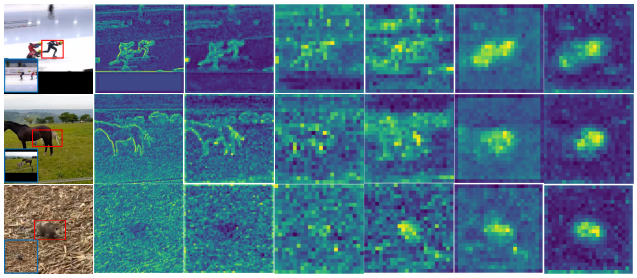


Figure 7. Visualization of spiking outputs of each MRM layer under three challenging scenarios (similar objects, occlusion, and background interference).

Template Expand Factor. Unlike previous methods, SpikeTrack adopts a template setting with the same size and expansion factor as the search region. Our experiments show that larger template expansion factors and higher template resolutions significantly improve accuracy, as presented in Tab. 3 (#7). We hypothesize that binary tensors lack fine-grained target detail, incorporating background information for contrastive representation allows for more global context and improves target encoding.

Loop Count in MRM. Figure 5 shows the results obtained with different retrieval loops in MRM. When the number of loops exceeds 1, we add a channel-wise learnable layer-scale on residual, to ensure training stability. One or two loops work best, while more can reduce performance due to accumulated errors and overly narrow focus.

Gap Analysis with Precision-Oriented Tracker. We compare SpikeTrack with the precision-oriented tracker OSTrack [37] across 14 attributes of the LaSOT dataset, as shown in the left panel of Fig. 6. A noticeable performance gap still exists between them. The right panel illustrates the average AUC gap between SpikeTrack-B variants and OSTrack-256. The largest gaps occur in the Deformation and Fast Motion scenarios, which pose greater challenges for deep semantic understanding and re-detection capabilities. We hope future SNN-based tracker designs can narrow the gap with ANN methods based on these insights. The full names of all attributes are provided in the Supplementary Material.

Visual Analysis. As shown in Fig. 7, we visualize the spiking outputs of each MRM layer under three challenging scenarios. It can be observed that MRM follows a global-to-instance perception process, building an understanding of the search region based on cues provided by memory. The method performs well under occlusion and background distraction, but in the similar-object interference scenario, although it ultimately locates the correct target, it is still affected by similar objects. We attribute this to the difficulty of representing fine-grained semantic information using spike-based encoding.

5. Conclusion

This work presents SpikeTrack, a family of spike-driven visual tracking models. With an asymmetric architecture and memory-retrieval-based unidirectional information transfer, SpikeTrack delivers energy-efficient and accurate RGB tracking. Extensive experiments demonstrate that SpikeTrack not only sets a new state-of-the-art among SNN-based trackers, but also shows competitive performance compared to recent ANN-based trackers, while significantly reducing energy consumption. We hope this work will advance research on SNN for RGB tracking and help narrow the gap with ANN-based trackers.

Limitations. A limitation of SpikeTrack lies in its difficulty handling scenes with similar objects. This is because the network does not have explicit modules for distinguishing similar objects, and spike information alone is insufficient to convey the fine-grained representations needed for such distinctions. In future work, we plan to build on this work to explore how to transmit fine-grained representations through spike-based mechanisms to tackle these challenging scenarios.

Acknowledgments

This work was supported in part by the NSFC under grant 62272344, in part by projects UID/04152/2025 and UID/PRR/04152/2025 from FCT (Fundação para a Ciência

e a Tecnologia) through the Centro de Investigação em Gestão de Informação (MagIC)/NOVA IMS.

References

[1] U Benchmark. A benchmark and simulator for uav tracking. In *ECCV*, 2016. 7

[2] Boyu Chen, Peixia Li, Lei Bai, Lei Qiao, Qiuhong Shen, Bo Li, Weihao Gan, Wei Wu, and Wanli Ouyang. Backbone is all your need: A simplified architecture for visual object tracking. In *ECCV*, pages 375–392. Springer, 2022. 1, 6, 7

[3] Xin Chen, Bin Yan, Jiawen Zhu, Dong Wang, Xiaoyun Yang, and Huchuan Lu. Transformer tracking. In *CVPR*, pages 8126–8135, 2021. 2, 5, 6, 7

[4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 5

[5] Heng Fan, Liting Lin, Fan Yang, Peng Chu, Ge Deng, Sijia Yu, Hexin Bai, Yong Xu, Chunyuan Liao, and Haibin Ling. Lasot: A high-quality benchmark for large-scale single object tracking. In *CVPR*, pages 5374–5383, 2019. 1, 5, 6

[6] Heng Fan, Hexin Bai, Liting Lin, Fan Yang, Peng Chu, Ge Deng, Sijia Yu, Harshit, Mingzhen Huang, Juehuan Liu, et al. Lasot: A high-quality large-scale single object tracking benchmark. *IJCV*, 129(2):439–461, 2021. 6

[7] Charlotte Frenkel, David Bol, and Giacomo Indiveri. Bottom-up and top-down approaches for the design of neuromorphic processing systems: Tradeoffs and synergies between natural and artificial intelligence. *Proceedings of the IEEE*, 111(6):623–652, 2023. 1

[8] Mark Horowitz. 1.1 computing's energy problem (and what we can do about it). In *2014 IEEE international solid-state circuits conference digest of technical papers (ISSCC)*, pages 10–14. IEEE, 2014. 6

[9] Lianghua Huang, Xin Zhao, and Kaiqi Huang. Got-10k: A large high-diversity benchmark for generic object tracking in the wild. *IEEE TPAMI*, 43(5):1562–1577, 2019. 5, 6

[10] Xiaoyang Jiang, Qiang Zhang, Jingkai Sun, Jiahang Cao, Jingtong Ma, and Renjing Xu. Fully spiking neural network for legged robots. In *ICASSP*, pages 1–5. IEEE, 2025. 2

[11] Ben Kang, Xin Chen, Dong Wang, Houwen Peng, and Huchuan Lu. Exploring lightweight hierarchical vision transformers for efficient visual tracking. In *ICCV*, pages 9612–9621, 2023. 6, 7

[12] Hei Law and Jia Deng. Cornernet: Detecting objects as paired keypoints. In *ECCV*, pages 734–750, 2018. 5

[13] Zhenxin Lei, Man Yao, Jiakui Hu, Xinhao Luo, Yanye Lu, Bo Xu, and Guoqi Li. Spike2former: Efficient spiking transformer for high-performance image segmentation. In *AAAI*, pages 1364–1372, 2025. 1, 2, 3, 7

[14] Liting Lin, Heng Fan, Zhipeng Zhang, Yong Xu, and Haibin Ling. Swintrack: A simple and strong baseline for transformer tracking. *NeurIPS*, 35:16743–16754, 2022. 6, 7

[15] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, pages 740–755. Springer, 2014. 5

[16] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017. 5

[17] Xinhao Luo, Man Yao, Yuhong Chou, Bo Xu, and Guoqi Li. Integer-valued training and spike-driven inference spiking neural network for high-performance and energy-efficient object detection. In *ECCV*, pages 253–272. Springer, 2024. 1, 2, 5, 7

[18] Yihao Luo, Min Xu, Caihong Yuan, Xiang Cao, Liangqi Zhang, Yan Xu, Tianjiang Wang, and Qi Feng. Siamsnn: Siamese spiking neural networks for energy-efficient object tracking. In *International conference on artificial neural networks*, pages 182–194. Springer, 2021. 1, 2, 6, 7

[19] Wolfgang Maass. Networks of spiking neurons: the third generation of neural network models. *Neural networks*, 10 (9):1659–1671, 1997. 3

[20] Wolfgang Maass. Networks of spiking neurons: the third generation of neural network models. *Neural networks*, 10 (9):1659–1671, 1997. 1

[21] Paul A Merolla, John V Arthur, Rodrigo Alvarez-Icaza, Andrew S Cassidy, Jun Sawada, Filipp Akopyan, Bryan L Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014. 1

[22] Matthias Muller, Adel Bibi, Silvio Giancola, Salman Alsubaihi, and Bernard Ghanem. Trackingnet: A large-scale dataset and benchmark for object tracking in the wild. In *ECCV*, pages 300–317, 2018. 5, 6

[23] Hamid Rezatofighi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *CVPR*, pages 658–666, 2019. 5

[24] Kaushik Roy, Akhilesh Jaiswal, and Priyadarshini Panda. Towards spike-based machine intelligence with neuromorphic computing. *Nature*, 575(7784):607–617, 2019. 1

[25] Catherine D Schuman, Shruti R Kulkarni, Maryam Parsa, J Parker Mitchell, Prasanna Date, and Bill Kay. Opportunities for neuromorphic computing algorithms and applications. *Nature Computational Science*, 2(1):10–19, 2022. 1

[26] Yimeng Shan, Zhenbang Ren, Haodi Wu, Wenjie Wei, RuiJie Zhu, Shuai Wang, Dehao Zhang, Yichen Xiao, Jieyuan Zhang, Kexin Shi, et al. Sdtrack: A baseline for event-based tracking via spiking neural networks. *arXiv preprint arXiv:2503.08703*, 2025. 1, 2, 7

[27] Hyeyoung Shin, Mora B Ogando, Lamiae Abdeladim, Uday K Jagadisan, Severine Durand, Ben Hardcastle, Hannah Belski, Hannah Cabasco, Henry Loefler, Ahad Bawany, et al. Recurrent pattern completion drives the neocortical representation of sensory inference. *Nature Neuroscience*, pages 1–11, 2025. 2, 4

[28] Zikai Song, Junqing Yu, Yi-Ping Phoebe Chen, and Wei Yang. Transformer tracking with cyclic shifting window attention. In *CVPR*, pages 8791–8800, 2022. 6

[29] Ning Wang, Wengang Zhou, Jie Wang, and Houqiang Li. Transformer meets tracker: Exploiting temporal context for robust visual tracking. In *CVPR*, pages 1571–1580, 2021. 6, 7

[30] Xiao Wang, Xiujun Shu, Zhipeng Zhang, Bo Jiang, Yaowei Wang, Yonghong Tian, and Feng Wu. Towards more flexible and accurate object tracking with natural language: Algorithms and benchmark. In *CVPR*, pages 13763–13773, 2021. 7

[31] Yi Wu, Jongwoo Lim, and Ming-Hsuan Yang. Online object tracking: A benchmark. In *CVPR*, pages 2411–2418, 2013. 7

[32] Shuiying Xiang, Tao Zhang, Shuqing Jiang, Yanan Han, Yahui Zhang, Xingxing Guo, Licun Yu, Yuechun Shi, and Yue Hao. Spiking siamfc++: Deep spiking neural network for object tracking. *Nonlinear dynamics*, 112(10):8417–8429, 2024. 1, 2, 7

[33] Bin Yan, Houwen Peng, Jianlong Fu, Dong Wang, and Huchuan Lu. Learning spatio-temporal transformer for visual tracking. In *CVPR*, pages 10448–10457, 2021. 5, 6, 7

[34] Jingjun Yang, Liangwei Fan, Jinpu Zhang, Xiangkai Lian, Hui Shen, and Dewen Hu. Fully spiking neural networks for unified frame-event object tracking. *arXiv preprint arXiv:2505.20834*, 2025. 1, 2, 7

[35] Man Yao, Jiakui Hu, Zhaokun Zhou, Li Yuan, Yonghong Tian, Bo Xu, and Guoqi Li. Spike-driven transformer. *NeurIPS*, 36:64043–64058, 2023. 2, 5, 7

[36] Man Yao, Xuerui Qiu, Tianxiang Hu, Jiakui Hu, Yuhong Chou, Keyu Tian, Jianxing Liao, Luziwei Leng, Bo Xu, and Guoqi Li. Scaling spike-driven transformer with efficient spike firing approximation training. *IEEE TPAMI*, 2025. 2, 4, 5, 7

[37] Botao Ye, Hong Chang, Bingpeng Ma, Shiguang Shan, and Xilin Chen. Joint feature learning and relation modeling for tracking: A one-stream framework. In *ECCV*, pages 341–357. Springer, 2022. 1, 2, 5, 6, 7, 8

[38] Weihao Yu, Mi Luo, Pan Zhou, Chenyang Si, Yichen Zhou, Xinchao Wang, Jiashi Feng, and Shuicheng Yan. Metaformer is actually what you need for vision. In *CVPR*, pages 10819–10829, 2022. 4

[39] Jiawen Zhu, Huayi Tang, Xin Chen, Xinying Wang, Dong Wang, and Huchuan Lu. Two-stream beats one-stream: asymmetric siamese network for efficient visual tracking. In *AAAI*, pages 10959–10967, 2025. 2, 6, 7

[40] Rui-Jie Zhu, Ziqing Wang, Leilani Gilpin, and Jason Eshraghian. Autonomous driving with spiking neural networks. *NeurIPS*, 37:136782–136804, 2024. 2

[41] Shihao Zou, Qingfeng Li, Wei Ji, Jingjing Li, Yongkui Yang, Guoqi Li, and Chao Dong. Spikevideoformer: An efficient spike-driven video transformer with hamming attention and o(t) complexity. *arXiv preprint arXiv:2505.10352*, 2025. 1, 2